UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MAR DEL PLATA

Tecnicatura Universitaria en Programación

Asignatura: Gestión de Desarrollo de Software

Trabajo Práctico Final

Dossier de Gestión del Proyecto

AlojaME — Sistema de Gestión y Reserva de Alojamientos

Integrante: Eloy Roldán

Docente a cargo: Eduardo Mango

Comisión: 2

Ciclo lectivo: 2026

Fecha de esta entrega: 11/11/2026 — Entrega y defensa final

Versión del documento: v1.0

Repositorio GitHub: github.com/EloyRolda/tp_gestion_alojamientos (rama FinalBranch)

Nota sobre esta entrega (v1.0)

Este documento corresponde a la entrega definitiva (versión 1.0) del dossier de gestión, prevista para el 11/11/2026 según el cronograma de la cátedra (segunda fecha: 18/11/2026). Incluye las 7 unidades completas —iniciación, interesados y alcance, arquitectura y descomposición, medición y estimación, riesgos y comunicación, ejecución y seguimiento, y cierre con retrospectiva y lecciones aprendidas— y refleja el estado del proyecto al 10/11/2026: backend auditado, frontend Angular completo y adaptado a celular, tablet y escritorio (Lote 5), y el estado de cada requisito funcional y no funcional (Anexo E). El formato de citas, referencias, tablas y figuras sigue las normas APA 7.ª edición (American Psychological Association, 2020).

Siguiendo el principio de la consigna de documentar lo que efectivamente ocurrió y mantener la trazabilidad, este dossier mantiene expuestas las brechas detectadas durante el proyecto: la pérdida del tablero Jira del cuatrimestre anterior, el desarrollo individual (el TP está pensado para equipos de 2 a 4 integrantes; el docente a cargo aceptó la modalidad individual) y la liquidación automática al anfitrión, que no puede completarse sin una habilitación especial de MercadoPago (requisito RF25 en estado parcial). Estas situaciones se documentan como riesgos y cambios en las secciones 5 y 6, con su respuesta, y se retoman críticamente en la retrospectiva de la sección 7.

Índice

1. Identificación e iniciación del proyecto.....5

1.1 Ficha de iniciación.....5

1.2 Misión, objetivos y propuesta de valor.....5

2. Definición del producto, interesados y alcance.....7

2.1 Interesados (stakeholders) y análisis poder–interés.....7

2.2 Requisitos funcionales y no funcionales priorizados (MoSCoW).....7

2.3 Alcance, exclusiones y límites.....9

2.4 MVP (producto mínimo viable).....10

2.5 Supuestos y restricciones.....10

2.6 Dependencias.....11

2.7 Entregables y líneas base.....11

3. Arquitectura y descomposición del trabajo.....13

3.1 Modelo C4.....13

3.2 EDT (Estructura de Desglose del Trabajo) y diccionario EDT.....16

3.3 Decisiones relevantes (ADR).....18

4. Medición, estimación y planificación integral.....19

4.1 PFA — Análisis de Puntos de Función.....19

4.2 COCOMO Básico.....20

4.3 Segunda técnica de estimación — Estimación por analogía.....21

4.4 Recursos, hitos y cronograma.....22

4.5 Valoración económica (ilustrativa).....23

5. Gestión de riesgos y comunicación.....25

5.1 Risk Register.....25

5.2 Plan de comunicación.....26

6. Ejecución, seguimiento y control.....28

6.1 Metodología de trabajo aplicada.....28

6.2 Evidencias de ejecución (historial de GitHub).....28

6.3 Cambios y desvíos registrados.....29

6.4 Planificado frente a ejecutado..... 31

6.5 Seguimiento de la integración frontend–backend–SQL.....32

7. Cierre, documentación final y lecciones aprendidas.....34

7.1 Cierre del proyecto..... 34

7.2 Documentación final..... 35

7.3 Retrospectiva.....35

7.4 Lecciones aprendidas.....36

7.5 Conclusiones finales.....37

Anexo A — Proyecto Jira.....38

Anexo B — Repositorios GitHub.....38

Anexo C — Diagrama entidad-relación de la base de datos.....39

Anexo D — Glosario..... 40

Anexo E — Matriz de trazabilidad de requisitos.....41

Referencias.....44

1. Identificación e iniciación del proyecto

1.1 Ficha de iniciación

Tabla 1. Ficha de iniciación del proyecto.

Campo	Detalle
Nombre del proyecto	AlojaME — Sistema de Gestión y Reserva de Alojamientos
Patrocinador / cátedra	UTN FRMdP — Gestión de Desarrollo de Software, en articulación con Programación IV
Fecha de inicio	29/08/2026 (preparación del esquema SQL de AlojaME)
Antecedente directo	Proyecto iniciado en Programación III como tp_gestion_alojamientos, en equipo (rama main del mismo repositorio; modelo con herencia Casa/Hotel/Departamento, sin pagos). En el ciclo 2026 fue reescrito íntegramente como AlojaME: nuevo modelo de dominio, pagos con MercadoPago, verificación de email, y una auditoría de arquitectura/seguridad consolidada en la versión final del backend.
Problema / necesidad	Propietarios de alojamientos turísticos y huéspedes carecen de una plataforma propia que centralice publicación, búsqueda, reserva y cobro, con trazabilidad de comisiones y sin que el anfitrión deba operar directamente la pasarela de pagos.
Equipo de trabajo	Eloy Roldán — desarrollo full-stack (backend Spring Boot y frontend Angular), arquitectura, gestión y documentación (proyecto individual; ver riesgo R2 en la sección 5)
Patrocinador académico	Docentes de Gestión de Desarrollo de Software, Programación IV y Metodología de Sistemas II — Comisión 2

1.2 Misión, objetivos y propuesta de valor

Misión:

Ofrecer una plataforma web que permita a anfitriones publicar y administrar alojamientos, y a clientes buscarlos, reservarlos y pagarlos de forma segura, centralizando en un único sistema la gestión de disponibilidad, pagos y reseñas.

Objetivo general:

Desarrollar e implementar el sistema AlojaME completo —API REST en Spring Boot, frontend Angular 22 y persistencia en una base SQL— que soporte el ciclo de vida de una reserva (publicación, búsqueda, reserva, pago, cancelación y reseña) para los roles cliente, anfitrión y administrador, gestionado con Scrum en Jira y versionado en GitHub, y que sirva de base para el Trabajo Final Integrador.

Objetivos específicos:

- Modelar el dominio completo (usuarios, alojamientos, reservas, pagos, reseñas) con integridad referencial y máquinas de estado explícitas.
- Integrar MercadoPago como pasarela de pagos bajo un modelo de retención y liquidación (Modelo B), con comisiones trazables.
- Garantizar autenticación y autorización por roles (CLIENTE, ANFITRIÓN, ADMIN) mediante JWT, incluyendo verificación de cuenta por correo.
- Auditar el código bajo principios SOLID y patrones de diseño (Strategy, DIP), eliminando deuda técnica detectada.
- Dejar una base de API estable (65 endpoints) para que el frontend Angular de Programación IV pudiera construirse sin bloqueos de backend.
- Construir el frontend Angular 22 con vistas por rol, guards e interceptor JWT, que cubra los 40 requisitos funcionales comprometidos y se adapte a celular, tablet y escritorio.

Propuesta de valor:

A diferencia del proyecto antecesor, AlojaME implementa un modelo de pagos con gross-up de comisiones (el cliente paga monto base + comisión de plataforma + comisión estimada de MercadoPago) donde el anfitrión nunca interactúa directamente con la pasarela: solo carga un alias/CBU/CVU validado. Esto reduce la fricción operativa del anfitrión y centraliza el riesgo de integración de pagos en un único componente, aislado mediante el patrón DIP (PasarelaDePagoService).

2. Definición del producto, interesados y alcance

2.1 Interesados (stakeholders) y análisis poder–interés

Tabla 2. Interesados del proyecto y análisis poder–interés.

Interesado	Rol frente al proyecto	Poder	Interés
Eloy Roldán	Desarrollador, arquitecto, gestor de proyecto (equipo completo)	Alto	Alto
Docente de Gestión de Desarrollo de Software	Evalúa el proceso de gestión; acceso de lectura a Jira/GitHub	Alto	Alto
Docente de Programación IV	Evalúa el software funcional (frontend + integración)	Alto	Alto
Docente de Metodología de Sistemas II	Evalúa la auditoría de código y diagramas UML	Alto	Medio
Clientes (usuario final)	Reservan y pagan alojamientos; dejan reseñas	Bajo	Alto
Anfitriones (usuario final)	Publican alojamientos y cobran vía alias/CBU	Bajo	Alto
Administrador (rol del sistema)	Modera usuarios, reservas y resuelve reembolsos manuales	Medio	Medio
MercadoPago (proveedor)	Pasarela de pagos; define límites de sandbox y SLA de API	Alto	Bajo
Brevo (proveedor)	Envío de emails de verificación	Medio	Bajo
Cloudinary (proveedor)	Almacenamiento y entrega de imágenes	Medio	Bajo

El análisis poder–interés sigue la matriz de interesados propuesta por la guía PMBOK (Project Management Institute [PMI], 2021). Los interesados de alto poder/alto interés (Eloy Roldán y los tres docentes) requieren gestión activa y comunicación frecuente (ver 5.2). Los proveedores externos, de alto poder pero bajo interés directo en el proyecto académico, se gestionan mediante el aislamiento arquitectónico (DIP) descrito en 3.3, que limita el impacto de un cambio en sus APIs.

2.2 Requisitos funcionales y no funcionales priorizados (MoSCoW)

Los requisitos se especifican siguiendo las recomendaciones de la práctica IEEE 830 (Institute of Electrical and Electronics Engineers [IEEE], 1998): cada uno es identificable, está priorizado y es verificable mediante un criterio de aceptación. La prioridad se asigna con la técnica MoSCoW (Clegg & Barker, 1994): Must (imprescindible para el MVP), Should (importante), Could (deseable) y Won't (fuera de esta entrega). La identificación (RF01 a RF40 y RNF01 a RNF15) es la misma de la Especificación de Requisitos (ERS) del proyecto, lo que permite seguir cada requisito entre las materias. La Tabla 3 resume los

requisitos por grupo funcional; el detalle individual, con su criterio de aceptación, estado e ítem de Jira que lo implementó, está en el Anexo E.

Tabla 3. Requisitos por grupo funcional: prioridad (MoSCoW), criterio de aceptación y estado.

Grupo	Requisitos (ERS)	Prioridad	Criterio de aceptación	Estado
Cuentas: registro, verificación por email, inicio de sesión y perfil	RF01–RF05	Must (RF01–RF03); Should (RF04–RF05)	Un usuario no puede iniciar sesión hasta confirmar el token enviado por email (24 h)	Hecho
Administración de usuarios	RF06–RF08	Should (RF06–RF07); Must (RF08)	Una cuenta desactivada deja de autenticar aunque tenga un JWT vigente	Hecho
Catálogos de tipos de alojamiento y servicios	RF09–RF10	Should	El administrador da de alta, renombra, fusiona y elimina sin duplicar nombres	Hecho
Alojamientos, galería y servicios asociados	RF11–RF17	Must (RF11, RF14, RF15); Should/Could (resto)	Un anfitrión gestiona solo sus propios alojamientos; las imágenes quedan accesibles por URL pública	Hecho (RF13 como baja lógica)
Reservas y su ciclo de vida	RF18–RF28	Must (RF18–RF22, RF25–RF27); Should (resto)	No se aceptan dos reservas activas con fechas superpuestas en el mismo alojamiento	Parcial: RF25 depende de la habilitación de liquidaciones en MercadoPago; el resto Hecho
Pagos con MercadoPago	RF29–RF33	Must (RF29–RF30); Should/Could (resto)	El pago pasa a PAGADO solo después de verificar su estado contra la API de MercadoPago	Hecho
Reseñas	RF34–RF35	Should	Una única reseña por reserva finalizada	Hecho
Estadísticas y resúmenes	RF36–RF38	Could	Rankings y totales calculados sin consultas N+1	Hecho
Búsqueda con filtros y comprobante imprimible	RF39–RF40	Could	Filtra por precio, capacidad y servicios; el comprobante se imprime sin elementos de navegación	Hecho
Ampliaciones para el TFI	RF-E1–RF-E4	Won't (en esta entrega)	Definidas y documentadas para la etapa siguiente	Pendiente (TFI)
Seguridad	RNF02, RNF05, RNF06	Must	Sin credenciales en el código; CORS limitado a orígenes explícitos	Hecho
Persistencia e integridad de datos	RNF03, RNF04, RNF14	Must	Base SQL; bloqueo pesimista en transiciones de estado; comisiones nunca reembolsadas	Hecho

Grupo	Requisitos (ERS)	Prioridad	Criterio de aceptación	Estado
Rendimiento	RNF07, RNF08	Should	Listados con JOIN FETCH o consultas agrupadas	Parcial: sin medición formal de tiempos de respuesta (RNF08)
Arquitectura y mantenibilidad	RNF10–RNF13	Should	Cada Service accede solo a su propio Repository; API documentada en Swagger	Hecho
Usabilidad, compatibilidad y adaptación a dispositivos	RNF01, RNF09, RNF15	Should	Sin desplazamiento horizontal y con controles táctiles de 44 px en 360, 768, 1024 y 1280 px	Hecho

2.3 Alcance, exclusiones y límites

Incluye (alcance final del proyecto integrador):

- Dominio completo de alojamientos, reservas, pagos y reseñas.
- Autenticación JWT con 3 roles y verificación de email.
- Integración con MercadoPago, Brevo y Cloudinary.
- 65 endpoints REST documentados con Swagger/OpenAPI.
- Frontend Angular 22 con vistas públicas y por rol (cliente, anfitrión y administrador), guards e interceptor JWT, que cubre los 40 requisitos funcionales comprometidos (RF01–RF40).
- Adaptación del frontend a celular, tablet y escritorio (Lote 5, RNF15).
- Persistencia en MariaDB/MySQL con esquema generado por JPA, más scripts de catálogo inicial y de datos de prueba.
- Paquete de despliegue para un servidor Linux con Nginx (frontend compilado, backend ejecutable, plantilla de configuración sin credenciales y manual de instalación).

Exclusiones (fuera del alcance de esta entrega):

- Mensajería directa entre anfitrión y cliente, notificaciones en tiempo real, cupones de descuento y respuesta del anfitrión a reseñas: son los requisitos RF-E1 a RF-E4, reservados para el TFI.
- Identidad visual definitiva (paleta de colores y tipografía de marca): el frontend se entrega con estética neutra y estructura responsive.
- Límite de intentos fallidos de inicio de sesión (protección contra fuerza bruta): gap conocido, aceptado y documentado (riesgo R9).
- Suite de tests automatizados: removida deliberadamente del backend (riesgo R8).

Límites conocidos:

- La liquidación automática al anfitrión (payouts) requiere que MercadoPago habilite esa función en la cuenta; sin ella, una reserva no puede pasar a FINALIZADA (RF25 parcial, riesgo R4).
- La notificación de pago (webhook) de MercadoPago solo llega a un dominio público con HTTPS; en el entorno local se verificó simulando la notificación.
- La dirección de un alojamiento no se edita después del alta (la API no expone esa operación).

2.4 MVP (producto mínimo viable)

El MVP se definió como: registro e inicio de sesión con verificación, publicación de un alojamiento, búsqueda y reserva, confirmación con pago vía MercadoPago, y cancelación con la política de reembolso correspondiente. El MVP está completo y verificado de punta a punta desde el frontend Angular contra el backend y una base MariaDB, incluido un pago completo en el entorno de prueba de MercadoPago.

2.5 Supuestos y restricciones

Tabla 4. Supuestos y restricciones.

Tipo	Descripción
Supuesto	Se asume disponibilidad de credenciales de prueba (sandbox) de MercadoPago, Brevo y Cloudinary durante todo el desarrollo.
Supuesto	Se asume que el frontend Angular (Programación IV) consumirá la API ya estabilizada sin requerir cambios de contrato. Resultado: se cumplió en lo esencial; el frontend solo requirió sumar alta y edición de catálogos y la renovación del token al cambiar el email (ver 6.3).
Restricción	El TP de Gestión exige equipos de 2 a 4 integrantes; este proyecto se ejecuta de forma individual, con la conformidad del docente a cargo (riesgo R2).
Restricción	ddl-auto=update de Hibernate no puede generar columnas GENERATED ni CHECK constraints; deben declararse vía anotaciones o SQL manual (documentado en la bitácora técnica).
Restricción	Con las credenciales de prueba, MercadoPago rechaza la API de liquidaciones (payouts) con el error "Invalid signature": requiere una habilitación de la cuenta que el proyecto no tiene. Preferencias de pago, consulta de pagos y webhook funcionan con las mismas credenciales.
Restricción	Condición técnica impuesta por la cátedra: frontend en Angular 22.
Restricción	Condición técnica impuesta por la cátedra: backend desarrollado en Spring Boot y comunicado con el frontend mediante HTTP.
Restricción	Condición técnica impuesta por la cátedra: autenticación y gestión de roles mediante JWT, con guards en el frontend.
Restricción	Condición técnica impuesta por la cátedra: persistencia en una base SQL real (sin JSON Server) y código versionado en GitHub.

Tipo	Descripción
Restricción	La versión de Angular usada exige Node.js 22.22.3 o superior (o 24.15+) para compilar.

2.6 Dependencias

Tabla 5. Dependencias del proyecto y riesgo asociado.

Dependencia	Tipo	Riesgo asociado
MercadoPago (Checkout Pro, Refunds, Payouts)	Externa, crítica	R4, R7
Brevo (envío de email transaccional)	Externa, crítica para el alta de usuarios	R7
Cloudinary (imágenes)	Externa, no bloqueante para el núcleo del negocio	R7
Angular 22 (Programación IV)	Interna al equipo; completada el 04/11/2026 y adaptada a dispositivos en el Lote 5	R5
MySQL/MariaDB 10.4+	Infraestructura	Gotcha documentado: columnas GENERATED pueden perderse en ALTER TABLE
Habilitación de liquidaciones (payouts) en MercadoPago	Externa, bloqueante para RF25	R4
Dominio público con HTTPS	Infraestructura, necesaria en producción para recibir el webhook de pagos	R10
Servidor Linux con Nginx, Java 21 y MariaDB	Infraestructura de despliegue (servidor propio del integrante)	R10

2.7 Entregables y líneas base

Tabla 6. Entregables y líneas base de alcance y tiempo.

Entregable	Línea base (alcance)	Línea base (tiempo)
Backend AlojaME (carpeta GA)	12 entidades, 65 endpoints, 3 integraciones externas	Checkpoint 1: 16/09/2026
Auditoría SOLID/patrones (Metodología II)	≥5 propuestas de mejora + diagramas UML	17/11/2026
Dossier de Gestión v0.9 (pre-entrega)	Unidades 1 a 6	28/10/2026
Dossier de Gestión v1.0	Unidades 1 a 7 + cierre	11/11/2026 (o 18/11, segunda fecha)
Frontend Angular (Programación IV)	40 RF (RF01–RF40) en 4 lotes por rol	Sprint del 27/10 al 04/11/2026 (entregado el 04/11/2026)

Entregable	Línea base (alcance)	Línea base (tiempo)
Frontend responsive (Lote 5)	9 historias, 36 puntos: celular, tablet y escritorio	Sprint del 10/11 al 17/11/2026
Paquete de despliegue y scripts SQL	Frontend compilado, backend ejecutable, Nginx, systemd, manual; catálogo inicial y datos de prueba	10/11/2026

3. Arquitectura y descomposición del trabajo

3.1 Modelo C4

Se documenta la arquitectura en tres niveles del modelo C4 (Brown, 2018): contexto, contenedores y componentes. El nivel de código no se incluye por redundar con el detalle de la auditoría SOLID/patrones (Metodología de Sistemas II).

Los tres diagramas respetan las convenciones de notación del modelo C4 (Brown, 2018): cada elemento declara su tipo y, en contenedores y componentes, su tecnología; las relaciones son unidireccionales, se dibujan con conectores ortogonales que no se cruzan y se rotulan con su propósito y, cuando corresponde, con el protocolo; y cada figura incluye su propia leyenda (Referencias). Como el documento se imprime en blanco y negro, el tipo de elemento se distingue por la forma y el borde, no por el color.

3.1.1 Diagrama de contexto

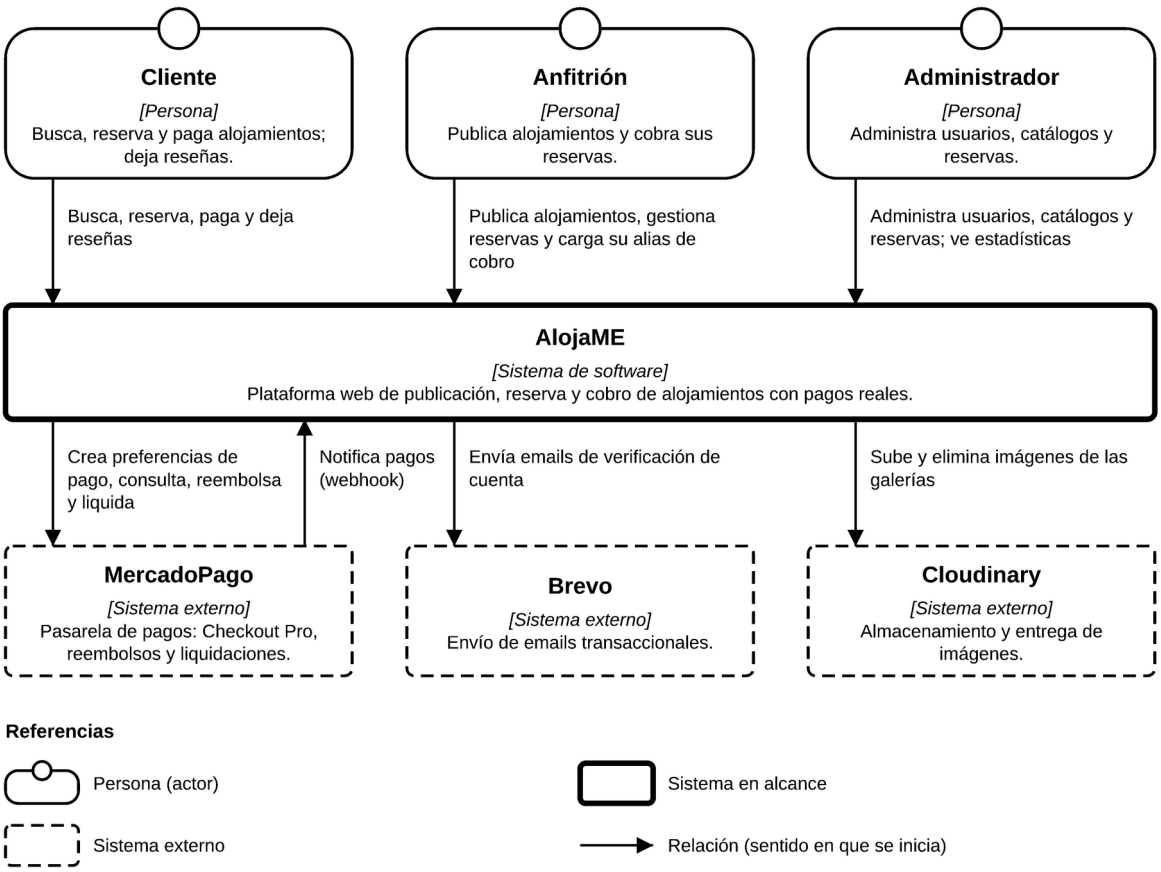


Figura 1. Diagrama de contexto C4 de AlojaME: actores, sistema en alcance y sistemas externos.

3.1.2 Diagrama de contenedores

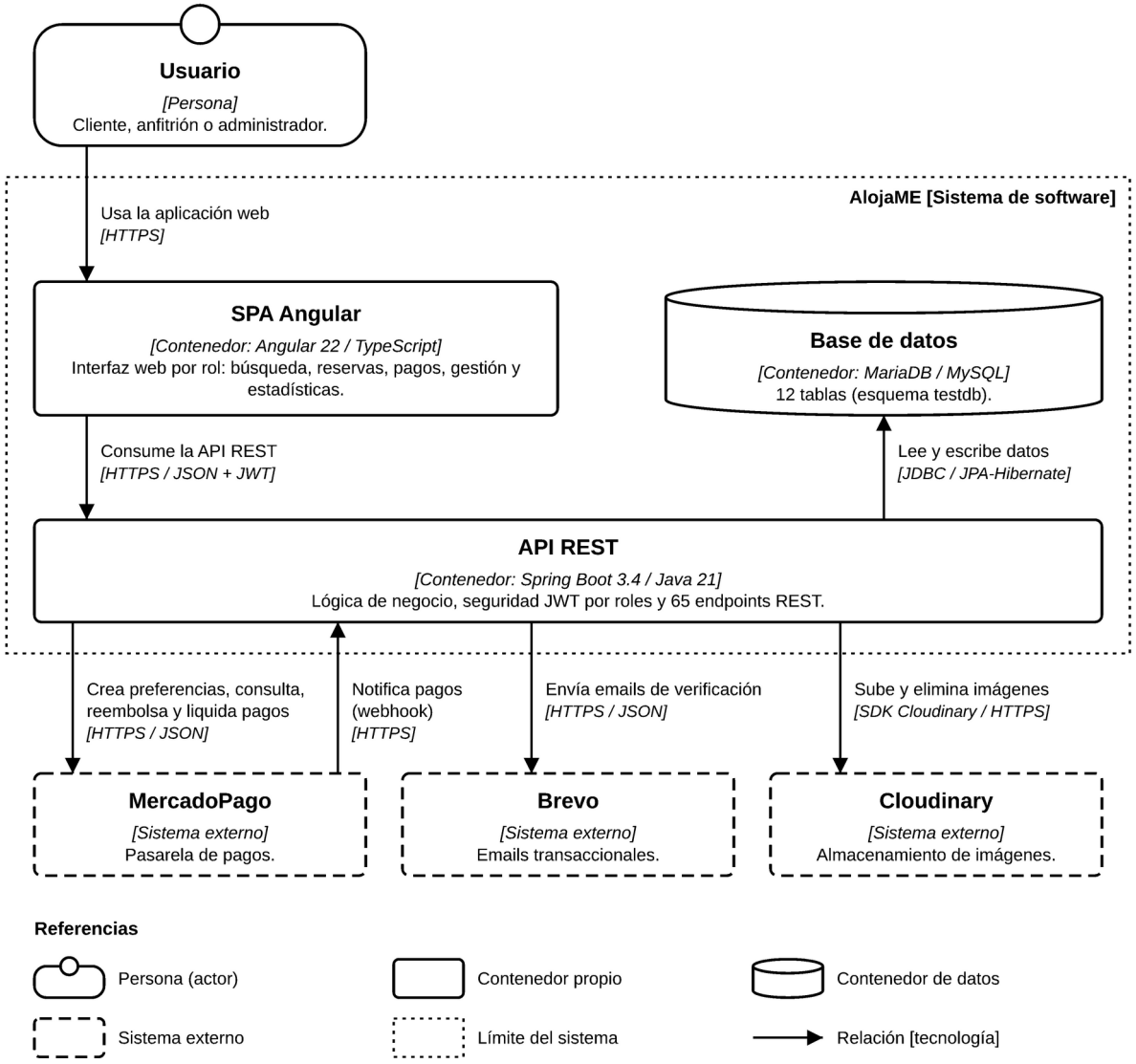


Figura 2. Diagrama de contenedores C4: SPA Angular, API REST y base de datos dentro del límite del sistema, con sus sistemas externos.

3.1.3 Diagrama de componentes

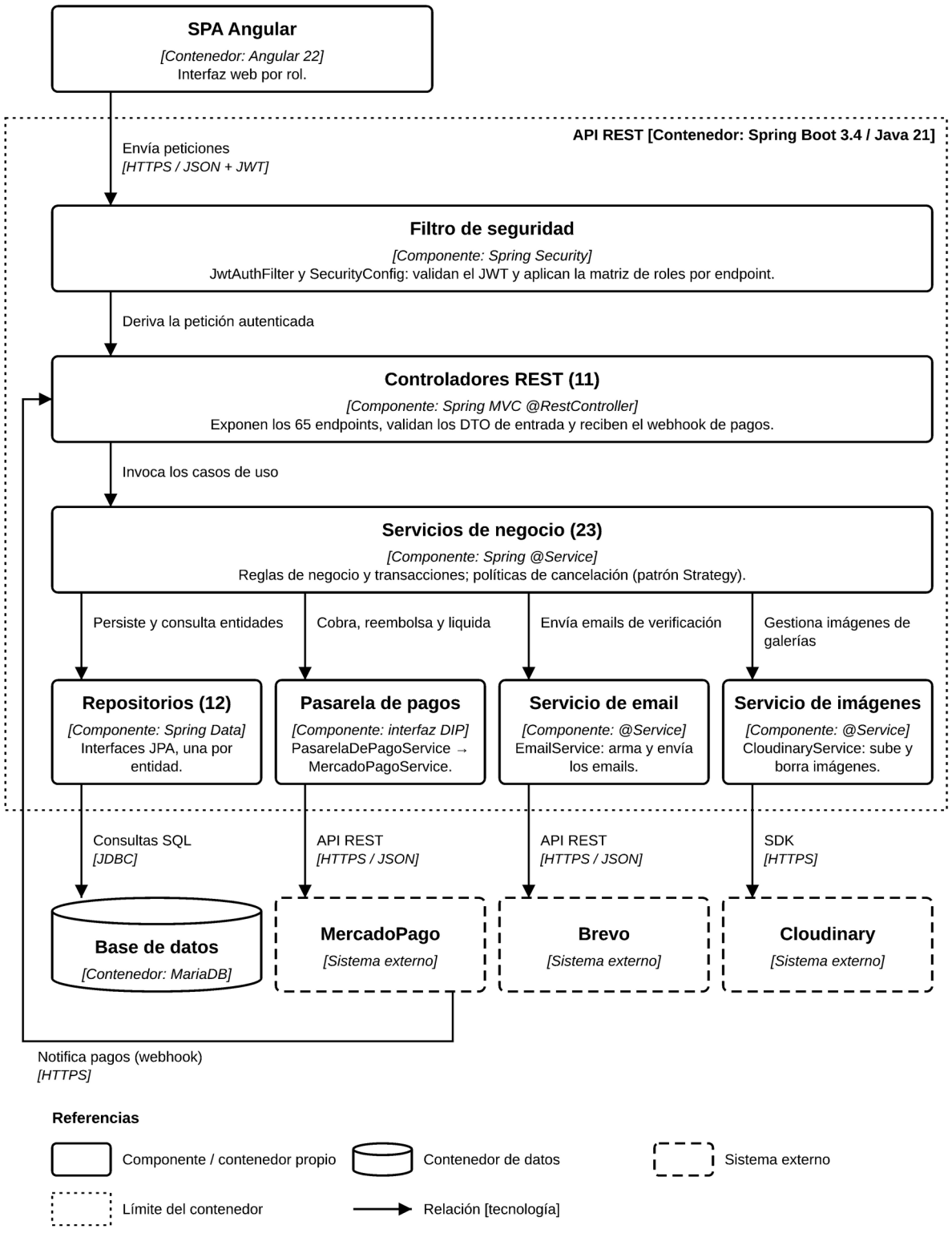


Figura 3. Diagrama de componentes C4 de la API REST: filtro de seguridad, controladores, servicios, repositorios y adaptadores a sistemas externos.

3.2 EDT (Estructura de Desglose del Trabajo) y diccionario EDT

La EDT se representa como un árbol jerárquico orientado a entregables (PMI, 2019): el nivel 0 es el proyecto, el nivel 1 agrupa las cuatro ramas y las hojas del árbol son los paquetes de trabajo, cuya numeración coincide con la del diccionario EDT (Tabla 7). Los conectores no llevan flechas porque expresan descomposición (todo–parte), no flujo.

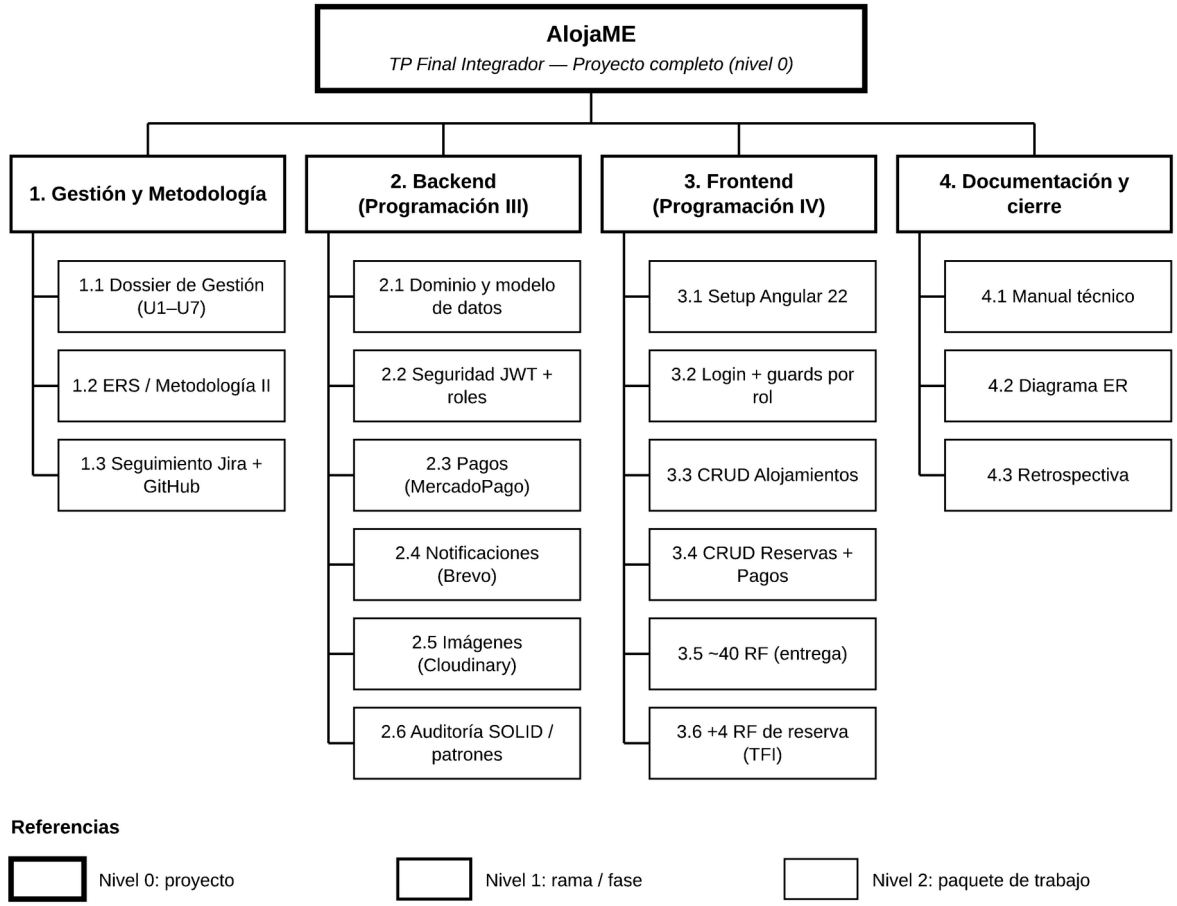


Figura 4. EDT de AlojaME: proyecto (nivel 0), 4 ramas (nivel 1) y 18 paquetes de trabajo (nivel 2).

Tabla 7. Diccionario EDT.

Código EDT	Entregable	Descripción	Criterio de aceptación
1.1	Dossier de Gestión (U1-U7)	Documento formal de gestión del proyecto	PDF conforme al formato de cátedra, aprobado en defensa
1.2	ERS / Metodología II	Auditoría de código + diagramas UML	≥5 propuestas de mejora fundamentadas
1.3	Seguimiento Jira + GitHub	Tablero Scrum y repositorio con historial	Acceso docente habilitado; trazabilidad tarea↔commit
2.1	Dominio y modelo de datos	12 entidades JPA, máquinas de estado Reserva/Pago	Esquema persistido y verificado en MariaDB

Código EDT	Entregable	Descripción	Criterio de aceptación
2.2	Seguridad JWT + roles	Autenticación stateless, 3 roles, verificación de email	JWT inválido/vencido no autentica; cuenta desactivada no autentica
2.3	Pagos (MercadoPago)	Checkout Pro, reembolsos, payouts, gross-up de comisiones	Flujo de pago probado end-to-end en sandbox
2.4	Notificaciones (Brevo)	Email de verificación con token de un solo uso (24 h)	Login bloqueado hasta confirmar el token
2.5	Imágenes (Cloudinary)	Carga y gestión de galerías de alojamientos	Imagen accesible vía URL pública tras la carga
2.6	Auditoría SOLID/patrones	Strategy (cancelación), DIP (pasarela de pago), locking pesimista	4 bugs de concurrencia/seguridad corregidos y documentados
3.1	Setup Angular 22	Proyecto standalone, rutas, cliente HTTP y configuración de la URL de la API	Compila sin errores ni advertencias y consume la API del backend
3.2	Login + guards por rol	Registro, verificación, login, guards, interceptor JWT, navbar por rol, páginas 403/404	Una ruta protegida redirige a /login o /403 según corresponda
3.3	CRUD Alojamientos	Vistas públicas y del anfitrión: listado, búsqueda, detalle, alta/edición, galería	Alta, edición y galería verificadas contra el backend y Cloudinary
3.4	CRUD Reservas + Pagos	Reserva, detalle, confirmación, cancelación, pago y comprobante	Un pago de prueba queda PAGADO en la interfaz
3.5	~40 RF (entrega)	Resto de las vistas por rol (perfil, estadísticas, administración) y adaptación responsive (Lote 5)	40 RF verificados; sin desplazamiento horizontal en 360 px
3.6	+4 RF de reserva (TFI)	RF-E1 a RF-E4 definidos y documentados para la etapa siguiente	Requisitos especificados en el ERS (Anexo E)
4.1	Manual técnico	README del repositorio y del frontend, bitácora técnica y manual de despliegue	Permiten instalar y ejecutar el sistema sin asistencia
4.2	Diagrama ER	Modelo entidad-relación generado desde el esquema de la base	Coincide con las 12 tablas de la base (Anexo C)
4.3	Retrospectiva	Retrospectiva, lecciones aprendidas y cierre	Incluida en la sección 7 de este dossier

3.3 Decisiones relevantes (ADR — Architecture Decision Records)

Tabla 8. Registro de decisiones de arquitectura (ADR).

ADR	Decisión	Justificación	Consecuencia
ADR-01	Strategy pattern para políticas de cancelación (PoliticaCancelacion)	Cada motivo de cancelación (reembolso total, retención tardía, reembolso manual admin) tenía su propia matemática de reembolso mezclada en PagoService	Agregar un motivo nuevo es agregar una clase; PagoService ya no crece con un switch
ADR-02	DIP sobre PasarelaDePagoService (interfaz)	PagoService y las políticas dependían directamente de MercadoPagoService	Cambiar de proveedor de pagos es agregar una implementación nueva, no reescribir el negocio
ADR-03	Bloqueo pesimista (PESSIMISTIC_WRITE) en transiciones de estado de Reserva	Dos requests concurrentes podían pasar juntas el chequeo de estado y disparar el Payout/reembolso dos veces	Se corrigieron 2 condiciones de carrera (alta de reserva y transición de estado)
ADR-04	Variables de entorno sin default para las 6 credenciales sensibles	application.properties tenía credenciales escritas en el código (DB, Cloudinary, JWT secret, Brevo, MercadoPago)	La app no arranca si falta una variable; se recomienda rotar credenciales si la rama llega a producción
ADR-05	CORS por allowlist explícita (cors.allowed-origins)	setAllowedOriginPatterns(List.of("*")) combinado con allowCredentials(true) permitía requests autenticados desde cualquier origen	Solo el frontend configurado puede operar con JWT desde el navegador
ADR-06	Frontend con componentes standalone y signals (Angular 22)	Evitar módulos y estado global innecesarios en una app de este tamaño	Cada vista declara sus dependencias; el estado de sesión vive en un único servicio
ADR-07	Búsqueda filtrada en el cliente	La API de listado no recibe parámetros de filtro	Filtrar es instantáneo; si el catálogo crece, conviene mover los filtros al backend
ADR-08	CSS propio mobile-first, sin librerías de componentes	Estética deliberadamente simple hasta definir la identidad visual	Utilidades reutilizables (grillas, formularios, tablas como tarjetas); la paleta se cambia en un solo lugar
ADR-09	URL de la API configurable en ejecución y Nginx como proxy	Desplegar sin recompilar y sin exponer el backend	El backend escucha solo en la propia máquina; Nginx bloquea los endpoints internos
ADR-10	Baja lógica de alojamientos (activar/desactivar)	Un borrado físico arrastraba reservas y pagos históricos	El anfitrión pausa una publicación sin perder su historial (variante de RF13)

4. Medición, estimación y planificación integral

Se aplican dos técnicas independientes de estimación —Análisis de Puntos de Función (PFA) y COCOMO Básico— más una estimación por analogía contra el proyecto antecesor, con el objetivo de contrastar resultados y analizar críticamente sus divergencias, tal como pide la consigna.

4.1 PFA — Análisis de Puntos de Función

Se relevaron los 65 endpoints REST del backend y se clasificaron según el manual de conteo del International Function Point Users Group (IFPUG, 2009) en Entradas Externas (EI, operaciones que mutan datos: POST/PUT/PATCH/DELETE), Consultas Externas (EQ, lecturas simples) y Salidas Externas (EO, lecturas con cálculo/derivación, ej. estadísticas). Se usaron pesos de complejidad promedio de la tabla IFPUG simplificada.

Tabla 9. Conteo de puntos de función por tipo de componente.

Componente	Cantidad	Peso (promedio)	Subtotal
EI — Entradas externas (POST/PUT/PATCH/DELETE)	31	4	124
EO — Salidas externas (reportes/estadísticas)	6	5	30
EQ — Consultas externas (lecturas simples)	28	4	112
ILF — Archivos lógicos internos (entidades/tablas)	12	10	120
EIF — Archivos de interfaz externa (MercadoPago, Brevo, Cloudinary)	3	7	21
PFSC — Puntos de función sin ajustar			407

Factor de ajuste: FA = 0,65 + 0,01 × ΣFI, donde ΣFI es la suma del grado de influencia (de 0 a 5) de las 14 características generales del sistema (IFPUG, 2009), detalladas en la Tabla 10. Con ΣFI = 42 (complejidad media-alta por las 3 integraciones externas y la seguridad JWT, sin distribución geográfica ni concurrencia masiva), FA = 0,65 + 0,42 = 1,07.

Tabla 10. Grado de influencia de las 14 características generales del sistema.

N.º	Característica	Influencia (0–5)	Justificación
1	Comunicación de datos	5	Frontend y backend se comunican por HTTP con una API REST
2	Procesamiento distribuido	3	SPA, API y servicios externos (pagos, email, imágenes)
3	Rendimiento	3	Listados sin consultas N+1; sin exigencias de tiempo real

N.º	Característica	Influencia (0–5)	Justificación
4	Configuración muy utilizada	2	Un solo servidor con Nginx y una base
5	Tasa de transacciones	2	Volumen bajo a moderado de reservas y pagos
6	Entrada de datos en línea	5	Todo el ingreso de datos se hace en línea desde la web
7	Eficiencia del usuario final	4	Vistas por rol, filtros y adaptación a dispositivos
8	Actualización en línea	4	Reservas, pagos y catálogos se actualizan en línea
9	Procesamiento complejo	4	Comisiones, políticas de cancelación, concurrencia y webhook
10	Reusabilidad	3	Servicios y componentes reutilizables; pasarela de pago aislada
11	Facilidad de instalación	2	Paquete de despliegue con manual; configuración manual
12	Facilidad de operación	2	Procesos automáticos (vencimiento de pagos); sin monitoreo avanzado
13	Múltiples sitios	0	Una única instalación
14	Facilidad de cambio	3	Catálogos administrables y configuración en ejecución
	Total ΣFI	42	

PFA = PFSC × FA = 407 × 1,07 ≈ 436 Puntos de Función.

Usando una productividad de referencia para Java de ≈53 LOC/PF (tablas históricas de Jones, 1996), el PFA proyecta ≈23.100 LOC. El sistema terminado suma ≈13.600 LOC propias (6.957 de Java en el backend y 6.616 de TypeScript, HTML y CSS en el frontend, medidas con wc -l sobre el código fuente, sin tests ni dependencias), es decir ≈1,7 veces menos que lo proyectado. La divergencia es esperable: las tablas históricas asumen más código repetitivo del que generan Spring Boot y Lombok, y el proyecto no incluye suite de tests. Como las funciones del frontend son las mismas que expone la API, el conteo de puntos de función no se duplica por tener dos capas.

4.2 COCOMO Básico

Se aplica COCOMO Básico (Boehm, 1981) en modo orgánico (equipo pequeño, dominio conocido, requisitos flexibles), con las líneas de código propias medidas al cierre. Los resultados se expresan en persona-mes, meses calendario y personas promedio; no se traducen a dinero.

Tabla 11. COCOMO Básico en modo orgánico: E = 2,4 × KLOC^1,05; T = 2,5 × E^0,38; personas = E / T.

Alcance medido	KLOC	Esfuerzo E (persona-mes)	Tiempo T (meses)	Personas promedio
Backend (Java)	6,96	≈ 18,4	≈ 7,6	≈ 2,4
Frontend (TypeScript, HTML, CSS)	6,62	≈ 17,5	≈ 7,4	≈ 2,4
Sistema completo	13,57	≈ 37,1	≈ 9,9	≈ 3,8

COCOMO Básico sobreestima el esfuerzo, la duración y el equipo: el sistema completo se construyó con 1 persona en ≈2,4 meses calendario (29/08 al 10/11/2026), con dedicación parcial. Como no se llevó un registro de horas, el esfuerzo se reconstruyó a partir de la dedicación habitual del integrante: unas 2 horas diarias al backend y 1 hora diaria al frontend, de 4 a 5 días por semana. Eso da ≈67 a 84 horas para el backend (29/08 al 27/10/2026) y ≈26 a 32 horas para el frontend (26/09 al 10/11/2026, incluido el aprendizaje de Angular): ≈93 a 116 horas, es decir ≈0,6 a 0,7 persona-mes (a 160 horas por persona-mes), contra 37,1 persona-mes estimados por COCOMO. La diferencia es esperable: las constantes de COCOMO se calibraron sobre proyectos industriales con documentación formal, pruebas y coordinación de equipo, mientras que este proyecto reutiliza conocimiento de dominio del antecesor (tp_gestion_alojamientos) y omite deliberadamente la suite de tests. El contraste completo entre lo estimado y lo real está en la Tabla 21 (sección 6.4).

4.3 Segunda técnica de estimación — Estimación por analogía

Tabla 12. Proyectos de referencia para la estimación por analogía.

Proyecto	LOC	Archivos	Duración real
tp_gestion_alojamientos (Programación III, antecesor)	3.787	70	≈ 1 cuatrimestre (desde cero)
AlojaME, backend (este proyecto)	6.957 (backend)	119	≈ 2 meses (29/08 al 27/10/2026)

El backend de AlojaME es ≈1,8 veces más grande que su antecesor en líneas de código. Si el antecesor, construido desde cero, tomó un cuatrimestre completo, un escalado lineal ingenuo proyectaría ≈7,2 meses. Al no partir de cero —se reutiliza conocimiento de dominio, el modelo mental de la arquitectura Spring Boot y parte del esquema de base de datos— se aplica un factor de reutilización estimado de 0,35, que proyecta ≈2,5 meses. Esta estimación es la que mejor se ajustó a la duración efectiva del backend (≈2 meses) y se adoptó como referencia de planificación por sobre COCOMO Básico. El frontend no tiene un antecesor comparable (tp_gestion_alojamientos no tenía interfaz web), por lo que se planificó por sprint y por lotes de vistas (sección 6.1).

4.4 Recursos, hitos y cronograma

Tabla 13. Recursos del proyecto.

Recurso	Detalle
Humano	Eloy Roldán — único integrante: desarrollo backend y frontend, arquitectura, gestión y documentación
Infraestructura	MariaDB local para desarrollo, repositorio GitHub público (tp_gestion_alojamientos), proyecto Jira (Scrum) y servidor Linux propio para el despliegue (Nginx, Java 21, MariaDB)
Servicios externos	Cuentas sandbox: MercadoPago (Checkout Pro/Refunds/Payouts), Brevo (email transaccional), Cloudinary (imágenes)
Herramientas de desarrollo	Java 21 y Maven, Node.js 24 y Angular CLI, navegador con herramientas de prueba automatizada de interfaz

Tabla 14. Hitos del proyecto y estado.

Hito	Fecha	Estado
Inicio del proyecto (preparación de esquema SQL)	29/08/2026	Cumplido
Checkpoint 1 de la cátedra (iniciación, alcance)	16/09/2026	Cumplido — coincide con el primer sprint de desarrollo
Sprint de implementación del dominio y seguridad	16/09 – 21/09/2026	Cumplido
Checkpoint 2 de la cátedra (arquitectura, estimación)	14/10/2026	No cumplido en fecha — ver desvío D-1 en la sección 6.4
Auditoría SOLID/patrones + hardening de seguridad (versión final del backend)	20/10/2026	Cumplido
Diagrama ER y cierre de documentación técnica	27/10/2026	Cumplido
Pre-entrega integradora v0.9	28/10/2026	Cumplido
Entrega de Programación IV (frontend Angular, 4 lotes)	27/10 – 04/11/2026	Cumplido — 4 lotes verificados en vivo y auditados (PLAN_LOTES_FRONTEND.md)
Paquete de despliegue y scripts SQL	10/11/2026	Cumplido
Sprint "Frontend responsive - Lote 5"	10/11 – 17/11/2026	En curso — implementación completa; historias cerrándose en Jira a medida que se integran
Entrega y defensa final de Gestión (v1.0)	11/11/2026 (segunda fecha: 18/11)	Documentación completa (v1.0); defensa oral pendiente
Entrega de Metodología de Sistemas II	17/11/2026	En curso (versión v0.1 del documento lista)

La Figura 5 muestra el cronograma del proyecto, reconstruido a partir de los sprints de Jira y de la bitácora técnica, frente a los hitos y entregas fijados por la cátedra, que funcionan como línea base temporal.

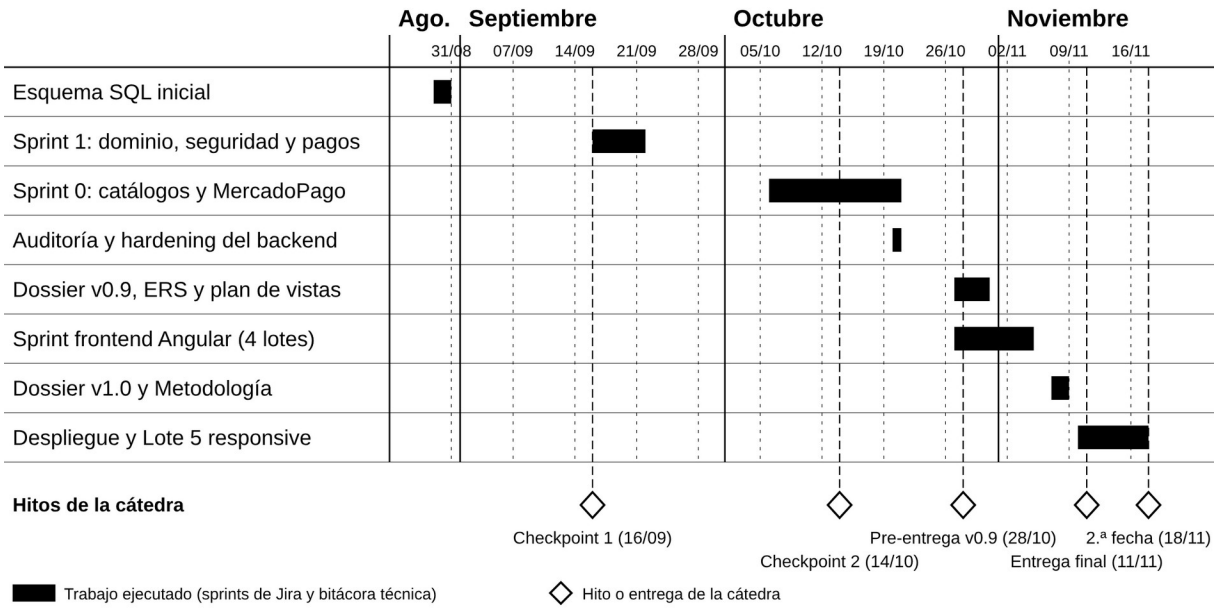


Figura 5. Cronograma del proyecto (diagrama de Gantt) frente a los hitos de la cátedra.

4.5 Valoración económica (ilustrativa)

La consigna pide valorar económicamente el proyecto. A fines comparativos —no como facturación, dado que se trata de un trabajo práctico académico— se valoran los recursos utilizados. Como no se llevó un registro de horas, el esfuerzo humano se reconstruye a partir de la dedicación habitual del integrante (sección 4.2) y se valoriza con una tarifa ilustrativa de USD 15 por hora. Las estimaciones de COCOMO se informan solo en persona-mes y no se convierten a dinero.

Tabla 15. Valoración económica ilustrativa de los recursos del proyecto.

Recurso	Base de cálculo	Valoración
Esfuerzo humano — backend (29/08 al 27/10/2026)	≈67 a 84 h (2 h diarias, 4 a 5 días por semana) × USD 15/h	USD 1.005 a 1.260
Esfuerzo humano — frontend (26/09 al 10/11/2026, incluido el aprendizaje de Angular)	≈26 a 32 h (1 h diaria, 4 a 5 días por semana) × USD 15/h	USD 390 a 480
Servicios externos (MercadoPago, Brevo, Cloudinary)	Cuentas de prueba y planes gratuitos	USD 0
Herramientas de gestión y versionado (Jira, GitHub)	Planes gratuitos	USD 0
Infraestructura de despliegue	Servidor propio del integrante; costo no imputado al proyecto	No imputado
Total	≈93 a 116 h	USD 1.395 a 1.740

La valoración muestra que el costo directo del proyecto académico es casi exclusivamente el tiempo del integrante: todos los servicios se usaron en sus modalidades gratuitas o de prueba. Para un uso comercial habría que sumar las comisiones efectivas de MercadoPago, el envío de emails por volumen y el alojamiento del servidor con dominio y HTTPS.

5. Gestión de riesgos y comunicación

5.1 Risk Register

El registro sigue la estructura de identificación, análisis cualitativo (probabilidad e impacto) y respuesta propuesta por la guía PMBOK (PMI, 2021). Se documentan los riesgos efectivamente identificados durante el desarrollo, incluyendo cinco que se materializaron (R1, R2, R3, R4 y R6), siguiendo el principio de la consigna de registrar la realidad del proyecto y no una versión idealizada.

Tabla 16. Registro de riesgos (Risk Register).

ID	Riesgo	Prob.	Impacto	Mitigación aplicada / planificada
R1	Pérdida de evidencia histórica de gestión: el proyecto Jira del cuatrimestre anterior fue eliminado automáticamente al pasar el tiempo entre cuatrimestres	Ocurrido	Alto	Se creó un nuevo proyecto Jira para el período de Gestión 2026; de aquí en más todo el trabajo se registra ahí en forma progresiva; el historial de GitHub y la bitácora técnica se usan como evidencia complementaria
R2	Equipo de un solo integrante frente a un TP pensado para 2-4 personas: bus factor = 1, sin par para revisión de código	Ocurrido	Alto	Priorización estricta de alcance (MVP primero); documentación exhaustiva (bitácora técnica y README) para permitir incorporar un integrante más adelante; 3 rondas de auditoría propia (arquitectura, seguridad y lógica de negocio). El docente a cargo aceptó el desarrollo individual
R3	Credenciales de servicios expuestas en el historial de git (escritas en el código en una versión anterior del backend)	Ocurrido	Alto	Las 6 credenciales ahora dependen de variables de entorno sin default (ADR-04); recomendación explícita de rotarlas si la rama llega a producción
R4	Limitaciones de las credenciales de prueba de MercadoPago: la API de liquidaciones (payouts) responde "Invalid signature" y, como finalizar una reserva incluye la liquidación, la reserva no puede pasar a FINALIZADA (RF25)	Ocurrido	Alto	Documentado como limitación de la cuenta, no del código. El resto del circuito de pagos se verificó con un pago completo de prueba. Respuesta propuesta para el TFI: liquidación manual registrada por el administrador (finalizar sin depender del payout y registrar la transferencia), con una vista de liquidaciones pendientes

ID	Riesgo	Prob.	Impacto	Mitigación aplicada / planificada
R5	Frontend Angular (Programación IV) sin iniciar a la fecha de la pre-entrega v0.9	Cerrado (04/11/2026)	Alto	Backend ya expone los 65 endpoints necesarios; planificación funcional (40 RF) ya relevada en el documento de Programación IV (Anexo B) para que el frontend arranque sin bloqueos de API. Cerrado: los 4 lotes del frontend se entregaron entre el 31/10 y el 04/11/2026
R6	Desfasaje entre el cronograma sugerido por la cátedra y el ritmo efectivo de trabajo (brecha entre el 21/09 y el 07/10/2026)	Ocurrido	Medio	Se documenta honestamente la brecha y su causa en la sección 6.4, en vez de reconstruir un historial artificial
R7	Dependencia de 3 proveedores externos (MercadoPago, Brevo, Cloudinary) fuera de control del equipo	Baja	Medio	Patrón DIP (PasarelaDePagoService) permite reemplazar el proveedor de pagos sin reescribir lógica de negocio; mismo criterio de aislamiento en EmailService/CloudinaryService
R8	Regresiones no detectadas por la ausencia de una suite de tests automatizados	Media	Medio	Recorridos de verificación completos por rol al cerrar cada lote, auditorías de código y verificación responsive automatizada en 4 anchos; tests de backend registrados como pendiente
R9	Ataques de fuerza bruta contra el inicio de sesión (no hay límite de intentos fallidos)	Baja	Medio	Riesgo aceptado para el alcance académico y documentado; respuesta definida: bloqueo temporal tras 5 intentos fallidos por usuario e IP
R10	Errores de configuración al desplegar en el servidor propio (credenciales, HTTPS, exposición del backend)	Media	Alto	Paquete de despliegue con plantilla sin credenciales, backend escuchando solo en la propia máquina, Nginx que bloquea endpoints internos, HTTPS con Certbot y recomendación de usar credenciales nuevas

5.2 Plan de comunicación

Tabla 17. Plan de comunicación.

Interesado	Medio	Frecuencia	Contenido
Docente de Gestión de Desarrollo de Software	Entregas versionadas (PDF) + acceso de lectura a Jira/GitHub	En cada checkpoint de la cátedra	Avance del dossier, acceso a evidencias
Docente de Programación IV	Repositorio GitHub + defensa oral	Según cronograma de esa materia	Avance del backend y, luego, del frontend

Interesado	Medio	Frecuencia	Contenido
Docente de Metodología de Sistemas II	Documento de auditoría + exposición oral	Entrega única, 17/11/2026	Propuestas de mejora y diagramas UML
Autocomunicación (equipo individual)	Commits descriptivos + tablero Jira + bitácora técnica	Continua	Decisiones de diseño, gotchas resueltos, estado de cada módulo

6. Ejecución, seguimiento y control

6.1 Metodología de trabajo aplicada

El trabajo se organizó con Scrum (Schwaber & Sutherland, 2020) en el proyecto Jira SCRUM, con épicas, historias y subtareas por bloque de trabajo. Como el tablero del cuatrimestre anterior se había perdido (R1) y el equipo es de un solo integrante, durante las primeras semanas el seguimiento diario se apoyó en commits descriptivos en un repositorio de trabajo y en una bitácora técnica de decisiones y errores resueltos. Jira se consolidó después en tres sprints: "SCRUM Sprint 0" (06/10 al 20/10/2026: catálogos como tablas, integración de MercadoPago y comprobante de pago); "Frontend Angular - Prog. IV" (27/10 al 04/11/2026: épica SCRUM-12, 5 historias y 31 subtareas, una por vista); y "Frontend responsive - Lote 5" (10/11 al 17/11/2026: épica SCRUM-49, 9 historias y 36 puntos de historia, con seguimiento por gráfico de burndown). Cada sprint parte de un objetivo explícito.

6.2 Evidencias de ejecución (historial de GitHub)

El repositorio oficial del proyecto es tp_gestion_alojamientos (Anexo B). Su rama main conserva la versión de Programación III, desarrollada en equipo entre el 05/05 y el 17/06/2026, y la rama FinalBranch contiene AlojaME: el backend en la carpeta GA y el frontend en la carpeta frontend. El backend se desarrolló entre el 29/08 y el 27/10/2026 en un repositorio de trabajo separado y se integró al repositorio oficial en un único commit el 27/10/2026; el frontend se integró por lotes (31/10, 06/11 y 10/11). Esto concentra el historial visible en pocos commits (desvío D-2, Tabla 20): la trazabilidad de esas etapas se documenta desde Jira (Tabla 19), y a partir del Lote 5 cada historia se integra en un commit propio que lleva su clave. La Tabla 18 resume la actividad de la rama FinalBranch.

Tabla 18. Actividad de la rama FinalBranch del repositorio oficial.

Fecha	Commits	Actividad principal
05/05 al 17/06/2026	57	Versión de Programación III (antecesor), desarrollada en equipo
27/10/2026	2	Integración del backend AlojaME (carpeta GA) y documentación de las materias
31/10/2026	1	Frontend Angular: infraestructura, vistas públicas y autenticación (Lote 1)
06/11/2026	1	Frontend Angular: vistas del rol cliente (Lote 2)
Desde el 10/11/2026	1 por historia, más 1 de integración	Lote 5 responsive (SCRUM-50 a SCRUM-58) e integración de las vistas de anfitrión y administrador (Lotes 3 y 4) con los ajustes del backend del 04/11

La integración automática entre Jira y GitHub no se configuró, así que la trazabilidad se documenta en la Tabla 19: cada ítem de Jira se vincula con el commit de la rama FinalBranch que lo incorporó, verificable con git show <hash>. Desde el Lote 5, el mensaje de cada commit empieza con la clave de su historia (por ejemplo, SCRUM-50).

Tabla 19. Trazabilidad entre ítems de Jira y commits de GitHub.

Ítem Jira	Trabajo realizado	Commit en FinalBranch	Fecha
SCRUM-2, 4, 6, 7, 9, 10 y 11 (Sprint 0)	Catálogos como tablas, integración de MercadoPago (checkout y payout), bloqueo pesimista y comprobante de pago	245a14e (integración del backend)	27/10/2026
SCRUM-13 y 14 (subtareas 18 a 27)	Infraestructura Angular, vistas públicas y autenticación	63b39bc	31/10/2026
SCRUM-15 (subtareas 28 a 34)	Vistas del rol cliente, con pago completo de prueba	95027e6	06/11/2026
SCRUM-16 y 17 (subtareas 35 a 48)	Vistas de anfitrión y administrador; alta y edición de catálogos en el backend	Commit de integración de los Lotes 3 y 4	10/11/2026
SCRUM-49 (historias 50 a 58)	Frontend responsive (Lote 5), una historia por commit	70c111b (SCRUM-50); SCRUM-51 a SCRUM-58: un commit cada una, identificado por su clave	Desde el 10/11/2026

6.3 Cambios y desvíos registrados

Tabla 20. Cambios y desvíos registrados.

Fecha	Cambio / desvío	Impacto	Resolución
20/10/2026	Se detectó que cancelar una reserva todavía SOLICITADA (sin confirmar, sin pago asociado) hacía fallar la operación en vez de cancelarla, reproducido en vivo contra MariaDB	Error funcional en una transición de estado válida según el modelo de dominio	Corregido: ReservaService solo invoca PagoService.ejecutarCancelacion cuando la reserva ya pasó por CONFIRMADA
20/10/2026	Auditoría de seguridad encontró credenciales escritas directamente en application.properties	Alto — credenciales expuestas en el historial de git	Las 6 credenciales pasaron a depender de variables de entorno sin default; documentado en la bitácora técnica con recomendación de rotación
20/10/2026	CORS con wildcard (*) combinado con allowCredentials(true)	Alto — cualquier sitio externo podía operar con JWT del usuario	Reemplazado por allowlist explícita (cors.allowed-origins)

Fecha	Cambio / desvío	Impacto	Resolución
20/10/2026	Dependencia faltante (micrometer-registry-prometheus) hacía que /actuator/prometheus devolviera 404 pese a estar documentado	Medio — observabilidad no funcional	Dependencia agregada al pom.xml
27/10/2026	El backend, desarrollado desde el 29/08 en un repositorio de trabajo separado, se integró al repositorio oficial en un único commit; los Lotes 3 y 4 del frontend se integraron recién el 10/11	Medio — la evolución commit a commit de esas etapas no se ve en el repositorio oficial (desvío D-2)	Trazabilidad documentada desde Jira (Tabla 19); desde el Lote 5, un commit por historia con su clave
04/11/2026	La API no permitía dar de alta ni renombrar tipos de alojamiento y servicios (detectado al construir el Lote 4)	Medio — dos requisitos del frontend (SCRUM-43/44) no se podían cumplir	Se agregaron alta y edición a ambos catálogos, restringidas al rol ADMIN
04/11/2026	El reenvío del email de verificación respondía distinto según el email existiera o no	Medio — permitía averiguar qué emails estaban registrados	Respuesta única para todos los casos
04/11/2026	Cambiar el propio email cerraba la sesión sin aviso	Medio — el token seguía atado al email anterior	El backend emite un token nuevo y el frontend lo reemplaza de forma transparente
09/11/2026	Cambio de alcance: la adaptación del frontend a celular, tablet y escritorio pasó de excluida a comprometida, para el despliegue en el servidor y la exigencia de Programación IV	Medio — nuevo requisito RNF15 y un sprint adicional (SCRUM-49, 36 puntos)	Sprint "Frontend responsive - Lote 5" planificado en Jira con 9 historias y criterios de aceptación medibles
10/11/2026	La auditoría responsive inicial detectó que el menú del administrador desbordaba la página (572 px en tablet) y 6 tablas que obligaban a desplazarse de costado en el celular	Medio — vistas de administración incómodas en pantallas chicas	Menú desplegable y tablas convertidas en tarjetas (SCRUM-51 y SCRUM-55); verificación final sin hallazgos en 132 casos
10/11/2026	Reservar sin sesión armaba una ruta de retorno inválida, y el login aceptaba cualquier destino de retorno	Bajo — redirección incorrecta tras iniciar sesión	Ruta corregida y login limitado a rutas internas de la aplicación (SCRUM-53)
10/11/2026	Preparación del despliegue en un servidor propio	Medio — la URL de la API estaba fija en el código compilado	URL configurable en ejecución, plantilla de configuración sin credenciales y sitio Nginx con bloqueo de endpoints internos (ADR-09)

Fecha	Cambio / desvío	Impacto	Resolución
10/11/2026	Se confirmó que, sin la habilitación de payouts, "Finalizar estadía" no completa la transición: la reserva queda CONFIRMADA, el cliente no puede dejar reseña y el administrador no ve liquidaciones pendientes	Alto — RF25 queda parcial (R4)	Registrado como limitación; propuesta de liquidación manual por el administrador para el TFI

6.4 Planificado frente a ejecutado

El Checkpoint 1 de la cátedra (16/09/2026) se cumplió en fecha: el primer sprint de desarrollo comienza exactamente ese día. El Checkpoint 2 (14/10/2026) no se cumplió en fecha: el siguiente bloque de trabajo registrado es del 07/10/2026 (anterior, dentro de margen) pero el siguiente salto de actividad ocurre recién el 20/10/2026, seis días después del checkpoint sugerido. Esta brecha (desvío D-1) se debe a que el foco de esa semana se puso en completar y endurecer la integración de pagos antes de iniciar la auditoría formal, lo cual desplazó la entrega de los artefactos de arquitectura (C4, EDT) hacia la pre-entrega v0.9 en lugar del Checkpoint 2. No se considera un riesgo activo: el trabajo técnico subyacente (pagos, seguridad) estaba igualmente dentro del alcance comprometido, solo se reordenó su secuencia frente a la documentación formal de gestión.

La Tabla 21 contrasta lo estimado y planificado con lo logrado en alcance, tiempo, esfuerzo y equipo, y explica cada desvío.

Tabla 21. Estimado frente a real y análisis de desvíos.

Dimensión	Estimado / planificado	Real	Desvío y análisis
Alcance funcional	40 RF para la entrega (RF01–RF40)	39 RF completos y 1 parcial (RF25)	RF25 depende de una habilitación externa de MercadoPago (R4); no es un incumplimiento del equipo
Alcance no funcional	14 RNF del ERS	13 completos, RNF08 parcial; se sumó RNF15 (responsive), completo	RNF08 no se midió formalmente; RNF15 fue un cambio de alcance aprobado (6.3)
Tiempo — backend	≈ 2,5 meses (analogía); ≈ 7,6 meses (COCOMO)	≈ 2 meses (29/08 al 27/10/2026)	La analogía acertó; COCOMO sobreestimó ≈3,8 veces
Tiempo — sistema completo	≈ 9,9 meses (COCOMO)	≈ 2,4 meses (29/08 al 10/11/2026)	COCOMO Básico no representa un desarrollo individual que reutiliza dominio

Dimensión	Estimado / planificado	Real	Desvío y análisis
Esfuerzo — backend	≈ 18,4 persona-mes (COCOMO)	≈ 0,4 a 0,5 persona-mes (67 a 84 h)	Horas reconstruidas a partir de la dedicación habitual (4.2)
Esfuerzo — sistema completo	≈ 37,1 persona-mes (COCOMO)	≈ 0,6 a 0,7 persona-mes (93 a 116 h)	COCOMO no representa un desarrollo individual con dedicación parcial
Equipo	≈ 3,8 personas promedio (COCOMO)	1 persona con dedicación parcial	Riesgo R2 materializado; mitigado con priorización MoSCoW
Sprint "Frontend Angular - Prog. IV"	27/10 al 04/11/2026	Vistas terminadas el 04/11/2026; sprint cerrado en Jira el 09/11/2026	Cierre administrativo tardío en Jira, sin desvío en la construcción
Checkpoint 2 de la cátedra	14/10/2026	Artefactos presentados en la v0.9 (28/10/2026)	Desvío D-1: se priorizó cerrar pagos y seguridad antes de documentar

6.5 Seguimiento de la integración frontend–backend–SQL

La integración entre el frontend Angular, la API Spring Boot y la base MariaDB se controló de forma explícita en cada lote, no solo al final: cada vista se dio por terminada recién cuando funcionaba contra el backend y una base MariaDB, nunca contra datos simulados. La Tabla 22 resume los controles aplicados y lo que se encontró.

Tabla 22. Controles de integración entre frontend, backend y base de datos.

Aspecto controlado	Cómo se verificó	Resultado
Contrato de la API	Cada llamada HTTP de los servicios Angular se cruzó con los endpoints del backend (auditoría del Lote 4.1)	Todo coincide; 2 endpoints sin uso justificado (baja lógica y pago embebido en la reserva)
Funcionalidad faltante	Construcción de cada vista contra la API del backend	Faltaban alta y edición de catálogos y la pantalla de alias de cobro: se agregaron (6.3)
Seguridad entre capas	CORS por lista explícita, JWT en cada pedido, cuenta desactivada y token vencido	Correcto; al cambiar el email se renueva el token sin cortar la sesión
Persistencia SQL	Recorridos por rol sobre MariaDB, revisando las tablas después de cada operación	Estados de reserva y pago consistentes; reseña única por reserva
Integraciones externas	Pago completo de prueba con MercadoPago, carga de imágenes en Cloudinary y emails con Brevo	Funcionan; payouts bloqueados por la cuenta (R4)
Despliegue	Paquete probado detrás de Nginx: rutas de Angular, proxy /api y bloqueo de endpoints internos	Correcto; el webhook de pagos requiere el dominio público con HTTPS

Aspecto controlado	Cómo se verificó	Resultado
Datos iniciales	Scripts de catálogo inicial y de datos de prueba ejecutados sobre una base nueva y recorridos por rol	Carga en una transacción; no se duplica si se ejecuta dos veces

7. Cierre, documentación final y lecciones aprendidas

7.1 Cierre del proyecto

A la fecha de esta entrega (v1.0), el alcance comprometido para Programación IV está cumplido: backend auditado, los 4 lotes del frontend Angular (vistas públicas, cliente, anfitrión y administrador) verificados en vivo contra el backend, la adaptación a celular, tablet y escritorio (Lote 5) y el paquete de despliegue. De los 40 requisitos funcionales, 39 están completos y uno es parcial (RF25, liquidación automática al anfitrión, bloqueada por la habilitación de MercadoPago). Quedan abiertos, por decisión de alcance y no por incumplimiento, la cobertura de tests automatizados de backend y la identidad visual definitiva.

Tabla 23. Entregables comprometidos frente a logrados.

Entregable	Comprometido	Logrado	Observación
Backend AlojaME	65 endpoints, 12 entidades, 3 integraciones externas	100%	Estado óptimo: auditado en 3 rondas (arquitectura, seguridad y lógica de negocio). Pendientes conocidos y documentados: liquidación automática (R4) y límite de intentos de inicio de sesión (R9).
Frontend Angular	40 RF (4 lotes: público, cliente, anfitrión y administrador)	100% funcional (39 RF completos y RF25 parcial)	Estado óptimo: los 4 lotes verificados en vivo contra el backend.
Auditoría de arquitectura y seguridad	Backend, frontend y lógica de negocio	100% (3 rondas, con errores corregidos)	Estado óptimo: incluyó correcciones de seguridad y de lógica de negocio (ver 6.3).
Documentación técnica	README y bitácora de decisiones	100%	Generalmente buena; se mantuvo como documento vivo a lo largo de toda la cursada.
Seguimiento en Jira	Backlog y tablero Scrum activos	100% (3 sprints con historial)	Fue el entregable más complejo de sostener (ver 7.3); quedó resuelto antes del cierre.
Tests automatizados de backend	Cobertura de los servicios clave	No iniciado	Sin intención de realizarlos en este alcance; decisión de priorización documentada (R8).
Responsive (Lote 5)	Celular, tablet y escritorio sin perder funcionalidad	100% implementado; verificado en 132 casos (33 pantallas × 4 anchos)	Historias cerrándose en Jira dentro del sprint; la paleta de colores definitiva queda para el TFI.

Entregable	Comprometido	Logrado	Observación
Paquete de despliegue	Instalable en un servidor Linux con Nginx	100%	Probado detrás de Nginx; incluye manual de instalación y scripts SQL.

El cierre formal de esta etapa queda sujeto a la defensa oral del dossier ante el docente a cargo de Gestión de Desarrollo de Software, Eduardo Mango, prevista para el 11/11/2026 según el cronograma de la cátedra.

7.2 Documentación final

La documentación del proyecto queda distribuida en los siguientes artefactos: este dossier de gestión (PDF, versionado de v0.1 a v1.0); el README del repositorio oficial (Anexo B), con la guía de instalación, la arquitectura y el estado del proyecto, y el README del frontend, con su estructura y convenciones; PLAN_LOTES_FRONTEND.md, con el detalle lote por lote del frontend, incluido el Lote 5; la bitácora técnica, llevada de forma progresiva, donde quedó registrada cada decisión de arquitectura y cada error resuelto con su fecha; y el paquete de instalación, con su manual de despliegue y los scripts SQL de catálogo inicial, datos de prueba y borrado de datos de prueba. El tablero Jira (Anexo A) complementa esta documentación con el historial de sprints, tareas y gráficos de seguimiento.

7.3 Retrospectiva

7.3.1 Qué funcionó mejor de lo esperado

Avanzar mediante sprints e historias de usuario permitió mantener un control más preciso sobre los requisitos funcionales a lo largo de toda la cursada, evitando que el alcance se desdibujara entre lotes.

7.3.2 Qué generó fricción

La principal fricción fue trabajar en solitario: el proyecto integra exigencias de varias materias en simultáneo, y la ausencia de un equipo con quien repartir tareas o contrastar decisiones exigió sostener sin apoyo tanto la carga técnica como la de gestión, en un contexto de poca disponibilidad horaria por compromisos laborales y personales. A esto se sumó una fricción puntual con Jira: la herramienta tiene una curva de aprendizaje considerable, y en un momento del proceso de configuración solicitó datos de tarjeta para continuar con una suscripción, lo cual generó dudas sobre si se trataba de un paso obligatorio. Esto retrasó la carga sistemática del tablero durante varias semanas, al punto

de que mantenerlo actualizado llegó a sentirse como una tarea que competía con el tiempo de desarrollo en lugar de ayudarlo. La mitigación fue la documentada en 6.1: sostener el seguimiento día a día con commits descriptivos y con la bitácora técnica, y formalizar Jira más tarde de lo ideal, recién a partir de octubre.

7.3.3 Qué haría distinto desde el día 1

Analizar los requisitos funcionales y el modelo de datos con más profundidad antes de comprometer sprints, y adoptar Jira de forma más pausada y controlada desde el inicio: a pesar de la fricción inicial, es una herramienta relativamente completa para llevar registro y visualizar qué sigue, y vale la pena invertir tiempo en incorporarla temprano en lugar de hacerlo sobre el final.

7.3.4 Técnica de gestión más y menos útil

De las técnicas aplicadas en la materia, el Risk Register fue la que mejor resultado dio en la práctica: dos de los riesgos registrados (R1 y R2, sección 5.1) terminaron materializándose tal como se había previsto, y tenerlos documentados con una mitigación de antemano evitó que esas situaciones se convirtieran en un problema no gestionado. En el otro extremo, COCOMO Básico fue la técnica que menos representó la realidad del proyecto: al estar calibrada sobre proyectos industriales de equipo, sobreestimó fuertemente tanto el esfuerzo como la cantidad de personas necesarias para un desarrollo individual de este tamaño (ver 4.2).

7.4 Lecciones aprendidas

- Desglose previo y auditorías sistemáticas: buena parte de las tareas arrancaron con un desglose inicial en checklists antes de pasar a la implementación, y cada lote cerró con una auditoría de integridad y seguridad del código apoyada en herramientas de análisis y asistencia al desarrollo, lo que redujo el tiempo entre detectar un problema y corregirlo.
- Validar el modelo de datos contra la realidad del negocio antes de escalar sobre él: al inicio de la cursada, la dimensión de cada promesa de sprint dependía enteramente del análisis de dificultad hecho sobre el modelo de datos vigente en ese momento; algunas tablas no resultaron escalables y no terminaban de ajustar al resto del sistema a medida que se agregaban reglas de negocio nuevas (el caso de reserva.precio_total como columna GENERATED, documentado en el gotcha #1 del proyecto, es un ejemplo concreto). La lección es que conviene validar el esquema contra los casos de uso concretos —no solo contra el modelo conceptual— antes de comprometer sprints

completos sobre él, para detectar la necesidad de un rediseño en etapas tempranas y no a mitad de la implementación.

- Separar el alcance funcional del alcance visual del frontend: decidir que el frontend de Programación IV priorizara primero los 40 RF permitió entregar las 4 vistas por rol completas y verificadas contra el backend. La adaptación a dispositivos (Lote 5) se ejecutó después, como un sprint propio con criterios de aceptación medibles (sin desplazamiento lateral y con controles táctiles en 4 anchos), sin competir por tiempo con la lógica de negocio.
- Registrar las horas desde el primer día: como no se llevó registro, el esfuerzo tuvo que reconstruirse a partir de la dedicación habitual (sección 4.2), con un margen amplio que debilita la comparación con las estimaciones.
- Validar temprano las dependencias externas de punta a punta: la limitación de la API de liquidaciones de MercadoPago se conocía desde el Lote 2, pero su efecto completo (la reserva no puede finalizar) recién se dimensionó al preparar el despliegue.
- Trabajar desde el primer día en el repositorio oficial: desarrollar el backend en un repositorio separado y subirlo después en un solo commit dejó sin evidencia visible la evolución de esa etapa (desvío D-2), algo que un commit por tarea desde el inicio habría evitado.

Recomendación a futuros equipos: definir y acotar al detalle los requisitos funcionales y el MVP con MoSCoW desde el primer sprint —qué debe tener el sistema, cómo debe comportarse y con qué tecnologías— reduce la ambigüedad que después se traduce en retrabajo, sobre todo en un proyecto que integra varias materias con exigencias propias.

7.5 Conclusiones finales

La misión y los objetivos planteados en 1.2 se cumplieron: el sistema final —backend auditado, frontend funcional y adaptado a dispositivos, base SQL y paquete de despliegue — corresponde a lo proyectado al inicio, con una única limitación externa documentada (RF25). El frontend demandó algo más de tiempo del previsto por una curva de aprendizaje técnica inicial, que se fue resolviendo a lo largo de los lotes sin afectar el cumplimiento de los requisitos funcionales.

Se considera que el proyecto está en condiciones de continuar como Trabajo Final Integrador. Cubre 40 requisitos funcionales desarrollados de forma individual (39 completos y 1 parcial), lo que demuestra capacidad de trabajo integral sobre un sistema completo (dominio, seguridad, pagos con MercadoPago, frontend y despliegue). Los 4 RF adicionales del TFI (RF-E1 a RF-E4) ya están definidos y se plantean como mejoras de la experiencia

de usuario integradas al resto del sistema; la liquidación manual al anfitrión es la primera mejora candidata para resolver RF25.

Anexo A — Proyecto Jira

El proyecto Jira del cuatrimestre anterior (vinculado a tp_gestion_alojamientos) fue eliminado automáticamente al pasar el tiempo entre cuatrimestres (riesgo R1). Para 2026 se creó el proyecto SCRUM, que al 10/11/2026 registra 57 ítems (épicas, historias, subtareas y tareas) organizados en tres sprints: "SCRUM Sprint 0" (cerrado), "Frontend Angular - Prog. IV" (cerrado) y "Frontend responsive - Lote 5" (activo hasta el 17/11/2026), con historial de estados, responsable, prioridad y puntos de historia.

Enlace al proyecto Jira: tpgestionalojamientosjira2.atlassian.net (proyecto SCRUM — "Yo, Eloy")

Acceso docente: habilitado con permisos de lectura sobre el proyecto, backlog, tablero, issues e historial (Eduardo Mango), conforme exige la consigna.

Anexo B — Repositorios GitHub

Tabla 24. Repositorios GitHub del proyecto.

Repositorio	Rama	Contenido	Enlace
tp_gestion_alojamientos	FinalBranch	AlojaME: backend Spring Boot (carpeta GA), frontend Angular 22 (carpeta frontend) y documentación de las materias	github.com/EloyRolda/tp_gestion_alojamientos/tree/FinalBranch
tp_gestion_alojamientos	main	Versión de Programación III (antecesor, hasta el 17/06/2026), desarrollada en equipo; las demás ramas del repositorio son de esa etapa	github.com/EloyRolda/tp_gestion_alojamientos/tree/main

Nota: el documento de planificación funcional de Programación IV (40 RF para la entrega más 4 RF adicionales, RF-E1 a RF-E4 para el TFI, con el detalle de vistas necesarias) se gestiona como entregable separado de la materia Programación IV, referenciado aquí por tratarse del mismo proyecto integrador.

Anexo C — Diagrama entidad-relación de la base de datos

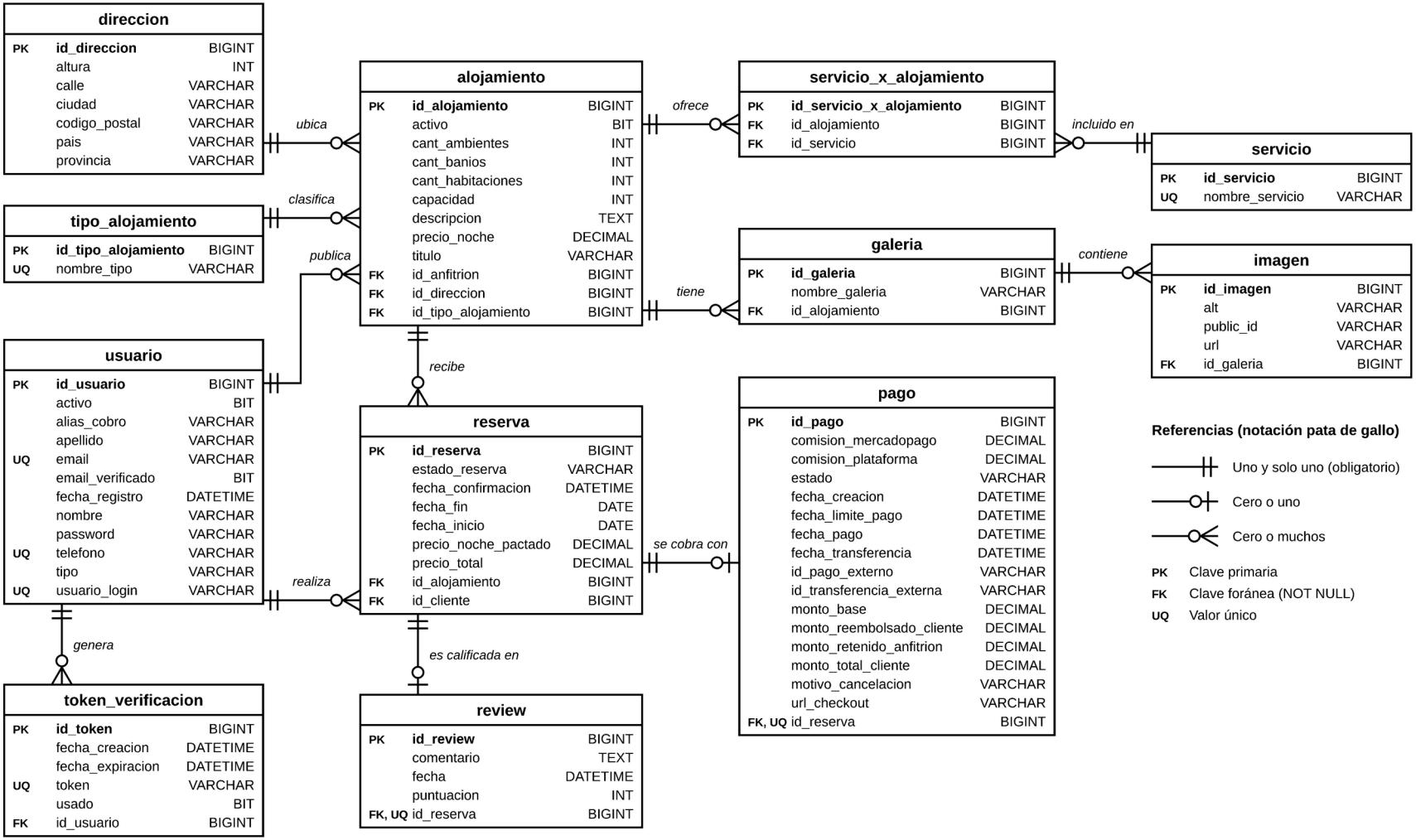


Figura 6. Diagrama ER de AlojaME (12 tablas) en notación pata de gallo (Everest, 1976), generado a partir del esquema de MariaDB.

Anexo D — Glosario

Tabla 25. Glosario de términos.

Término	Significado
ESTADORESERVA	Enum de estado de una Reserva: SOLICITADA, CONFIRMADA, FINALIZADA, CANCELADA
ESTADOPAGO	Enum de estado de un Pago: PENDIENTE, PAGADO, VENCIDO, REEMBOLSADO_TOTAL, REEMBOLSADO_PARCIAL
MOTIVOCANCELACION	Enum que determina la política de reembolso aplicada al cancelar una reserva
DTO	Data Transfer Object — objeto usado para exponer datos por la API sin acoplar al modelo JPA
JWT	JSON Web Token — mecanismo de autenticación stateless usado por el backend
DIP	Dependency Inversion Principle — principio SOLID aplicado en PasarelaDePagoService
Modelo B (de pagos)	La plataforma retiene el pago del cliente y liquida al anfitrión en un segundo momento
Gross-up	Técnica por la cual el cliente paga el monto base más las comisiones, para que el anfitrión reciba el neto esperado
PFA	Puntos de Función Ajustados — técnica de estimación de tamaño funcional de software
EDT / WBS	Estructura de Desglose del Trabajo (Work Breakdown Structure)
MoSCoW	Técnica de priorización: Must, Should, Could y Won't (debe, debería, podría, no en esta entrega)
Sprint	Iteración de Scrum con duración y objetivo fijos
Puntos de historia	Unidad relativa de estimación del esfuerzo de una historia de usuario
Burndown	Gráfico del trabajo restante de un sprint a lo largo del tiempo
Payout	Transferencia de la plataforma al anfitrión (liquidación) a través de MercadoPago
Responsive	Interfaz que se adapta al ancho de la pantalla (celular, tablet, escritorio)

Anexo E — Matriz de trazabilidad de requisitos

Estado de cada requisito al 10/11/2026, con su prioridad MoSCoW, criterio de aceptación e ítem de Jira que lo implementó. Los requisitos se identifican igual que en la ERS. "Hecho" significa implementado y verificado contra el backend y la base de datos; "Parcial", implementado con una limitación documentada; "Pendiente (TFI)", definido para la etapa siguiente.

Tabla 26. Matriz de trazabilidad de requisitos funcionales.

ID	Requisito	Prior.	Criterio de aceptación	Estado	Jira
RF01	Registro como cliente o anfitrión	Must	El alta queda pendiente de verificación	Hecho	SCRUM-25
RF02	Verificación de cuenta por token de un solo uso (24 h)	Must	Sin verificar no se puede iniciar sesión	Hecho	SCRUM-26
RF03	Inicio de sesión con JWT	Must	Credenciales válidas devuelven un JWT	Hecho	SCRUM-27
RF04	Consultar la información personal	Should	Muestra los datos del usuario logueado	Hecho	SCRUM-28
RF05	Modificar la información personal y el alias de cobro	Should	Los cambios persisten; el alias se valida antes de guardarse	Hecho	SCRUM-28
RF06	Administrador: consultar usuarios	Should	Listado con filtro por tipo	Hecho	SCRUM-42
RF07	Administrador: desactivar usuarios	Should	Una cuenta desactivada no inicia sesión	Hecho	SCRUM-42
RF08	Un usuario desactivado no autentica con un JWT previo	Must	El token vigente es rechazado	Hecho	Auditoría del 20/10/2026
RF09	Catálogo de tipos de alojamiento (alta, edición, baja)	Should	No se duplican nombres; se pueden fusionar	Hecho	SCRUM-2, SCRUM-43
RF10	Catálogo de servicios (alta, edición, baja)	Should	No se borra un servicio en uso	Hecho	SCRUM-4, SCRUM-44
RF11	Publicar un alojamiento con tipo y dirección	Must	Queda visible en la búsqueda	Hecho	SCRUM-35, SCRUM-36
RF12	Modificar un alojamiento propio	Should	Solo el dueño o el administrador	Hecho	SCRUM-36
RF13	Eliminar un alojamiento propio	Could	Sin reservas activas	Hecho (como baja lógica, ADR-10)	SCRUM-35
RF14	Consultar alojamientos disponibles	Must	Accesible sin iniciar sesión	Hecho	SCRUM-22, SCRUM-23
RF15	Ver el detalle de un alojamiento	Must	Muestra galería, servicios y reseñas	Hecho	SCRUM-24
RF16	Gestionar la galería de imágenes (Cloudinary)	Should	Imagen accesible por URL pública; máximo 10	Hecho	SCRUM-37
RF17	Asociar y quitar servicios de un alojamiento	Should	Los cambios se reflejan en el detalle	Hecho	SCRUM-36
RF18	Solicitar una reserva por rango de fechas	Must	La reserva queda SOLICITADA	Hecho	SCRUM-29

ID	Requisito	Prior.	Criterio de aceptación	Estado	Jira
RF19	Validar que no haya fechas superpuestas	Must	Rechaza reservas activas superpuestas, incluso concurrentes	Hecho	SCRUM-29, SCRUM-10
RF20	Confirmar una reserva (anfitrión o administrador)	Must	Pasa a CONFIRMADA y abre el plazo de pago de 48 h	Hecho	SCRUM-39
RF21	Generar el pago y la preferencia de MercadoPago al confirmar	Must	Pago PENDIENTE con su enlace de pago	Hecho	SCRUM-6, SCRUM-9
RF22	El cliente cancela su reserva	Must	Reembolso según anticipación (100% o 65%)	Hecho	SCRUM-31
RF23	El anfitrión cancela una reserva	Should	Reembolso del 100% al cliente	Hecho	SCRUM-39
RF24	El administrador cancela con reembolso manual	Should	Monto menor o igual al monto base	Hecho (verificado hasta la API de reembolsos)	SCRUM-45, SCRUM-46
RF25	Finalizar la reserva disparando la liquidación al anfitrión	Must	Pasa a FINALIZADA y se transfiere el monto base	Parcial: la liquidación requiere la habilitación de payouts (R4)	SCRUM-39, SCRUM-9
RF26	El cliente consulta sus reservas	Must	Listado con estado y total	Hecho	SCRUM-30
RF27	El anfitrión consulta las reservas recibidas	Must	Listado con filtro por estado	Hecho	SCRUM-38
RF28	Cancelación automática si no se paga en 48 h	Should	El pago pasa a VENCIDO y la reserva a CANCELADA	Hecho	Proceso programado del backend
RF29	Pagar con MercadoPago (Checkout Pro)	Must	Un pago aprobado deja la reserva pagada	Hecho	SCRUM-31, SCRUM-32
RF30	Procesar la notificación (webhook) de MercadoPago	Must	Consulta el estado del pago en la API antes de actualizarlo	Hecho (en local, con notificación simulada)	SCRUM-6
RF31	Consultar el detalle de un pago	Should	Solo el cliente, el anfitrión o el administrador involucrados	Hecho	SCRUM-11, SCRUM-31
RF32	Flujo de dinero global (administrador)	Could	Facturación, comisiones, liquidado y reembolsado	Hecho	SCRUM-47
RF33	Cobros pendientes del anfitrión	Should	Reservas pagadas aún no liquidadas	Hecho	SCRUM-40
RF34	Dejar una reseña de una reserva finalizada	Should	Una única reseña por reserva	Hecho	SCRUM-33
RF35	Consultar las reseñas de un alojamiento	Should	Visibles sin iniciar sesión	Hecho	SCRUM-24
RF36	Estadísticas globales (administrador)	Could	Rankings, cancelaciones por motivo, ingresos mensuales	Hecho	SCRUM-47, SCRUM-48
RF37	Resumen de alojamientos del anfitrión	Could	Ingresos, reservas y rating por alojamiento	Hecho	SCRUM-41
RF38	Resumen de gastos del cliente	Could	Total y promedio por estadía	Hecho	SCRUM-34
RF39	Búsqueda con filtros de precio, capacidad y servicios	Could	Filtros combinables (ADR-07)	Hecho	SCRUM-23

ID	Requisito	Prior.	Criterio de aceptación	Estado	Jira
RF40	Comprobante de pago imprimible	Could	Vista de impresión sin navegación	Hecho	SCRUM-11, SCRUM-31
RF-E1	Mensajería entre cliente y anfitrión por reserva	Won't	Definido en la ERS	Pendiente (TFI)	—
RF-E2	Notificación en tiempo real de nuevas solicitudes	Won't	Definido en la ERS	Pendiente (TFI)	—
RF-E3	Cupón de descuento sobre el monto base	Won't	Definido en la ERS	Pendiente (TFI)	—
RF-E4	Respuesta pública del anfitrión a una reseña	Won't	Definido en la ERS	Pendiente (TFI)	—

Tabla 27. Matriz de trazabilidad de requisitos no funcionales.

ID	Requisito	Prior.	Estado	Evidencia
RNF01	Interfaz web intuitiva y fácil de usar	Should	Hecho	SCRUM-12, SCRUM-49
RNF02	Autenticación JWT que invalida cuentas desactivadas	Must	Hecho	Auditoría del 20/10/2026
RNF03	Persistencia en base relacional (MySQL/MariaDB)	Must	Hecho	Esquema JPA (Anexo C)
RNF04	Integridad ante operaciones concurrentes (bloqueo pesimista)	Must	Hecho	SCRUM-10
RNF05	Sin credenciales en el código fuente	Must	Hecho	ADR-04
RNF06	CORS restringido a orígenes explícitos	Must	Hecho	ADR-05
RNF07	Listados sin consultas N+1	Should	Hecho	Auditoría del 20/10/2026
RNF08	Tiempos de respuesta adecuados	Should	Parcial: sin medición formal	—
RNF09	Accesible desde navegadores modernos	Should	Hecho (verificado en Chromium)	SCRUM-58
RNF10	Arquitectura en capas (cada Service con su Repository)	Should	Hecho	Auditoría del 20/10/2026
RNF11	API documentada con Swagger / OpenAPI	Should	Hecho	Backend
RNF12	Validación de datos de entrada (Bean Validation)	Should	Hecho	Backend
RNF13	Principios SOLID y patrones (Strategy, DIP)	Should	Hecho	ADR-01, ADR-02
RNF14	Las comisiones nunca se reembolsan	Must	Hecho	SCRUM-6
RNF15	Adaptación a celular, tablet y escritorio, sin desplazamiento lateral y con controles táctiles de 44 px (incorporado el 09/11/2026)	Should	Hecho (verificado en 360, 768, 1024 y 1280 px)	SCRUM-49 a SCRUM-58

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Boehm, B. W. (1981). *Software engineering economics*. Prentice-Hall.

Brown, S. (2018). *The C4 model for visualising software architecture*. https://c4model.com

Clegg, D., & Barker, R. (1994). *Case method fast-track: A RAD approach*. Addison-Wesley.

Everest, G. C. (1976). Basic data structure models explained with a common example. En *Proceedings of the Fifth Texas Conference on Computing Systems* (pp. 39–46). IEEE Computer Society.

Institute of Electrical and Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications* (ANSI/IEEE Std 830-1998).

International Function Point Users Group. (2009). *Function point counting practices manual* (release 4.3.1).

Jones, C. (1996). *Applied software measurement: Assuring productivity and quality*. McGraw-Hill.

Project Management Institute. (2019). *Practice standard for work breakdown structures* (3.ª ed.).

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK guide)* (7.ª ed.).

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html